Problem A. Iridescent Universe 2

Binary search the answer d, where $0 < d < \pi$. For a fixed d, expand each great-circle segment by distance d on both sides. This gives n spherical "sausages". We need to determine whether there exists a point that lies in the interior of at most k of them.

The boundaries of these sausages partition the sphere into regions. It is enough to check points on the boundaries of the arrangement. If $d < \pi/2$, the boundary of each sausage consists of four small-circle arcs; otherwise it consists of two. Extract all small circles, enumerate their intersection points, and test how many sausages contain each point. This yields an $O(n^3 \log(1/\varepsilon))$ algorithm.

A faster expected implementation comes from randomization. Enumerate sausages in random order and binary search only on a sausage when it can improve the current answer. In expectation, only $O(\log n)$ sausages cause improvements, leading to

$$O(n^3 + n^2 \log n \log(1/\varepsilon))$$

time. A sweep-line algorithm on the sausage boundaries can further reduce the theoretical complexity, but is much more difficult to implement.

Problem B. Queue Editor

No analysis was provided in the source editorial at the time used.

Problem C. Adjacent Add

The following quantity is invariant under the operation:

$$C = A_1 - \frac{1}{K}A_2 + \frac{1}{K^2}A_3 + \cdots + \left(-\frac{1}{K}\right)^{N-1} A_N.$$

If all entries can be made equal to x, then this invariant must also equal

$$x - \frac{1}{K}x + \frac{1}{K^2}x + \cdots + \left(-\frac{1}{K}\right)^{N-1} x.$$

Subtracting the two expressions gives a linear equation $G(x) = 0$. Since $K \geq 2$, $G(x)$ is strictly increasing, so it has exactly one real root.

Therefore the task is to decide whether this root is an integer. It is enough to binary search over $|x| \leq 2 \times 10^9$. For a fixed x, the sign of $G(x)$ can be computed in $O(N)$ time with careful comparison to avoid precision issues.

The time complexity is $O(N \log \max A)$.

Problem D. Circular Matching

First solve the subproblem where $2M$ red and blue balls are evenly placed on a circle and we need the minimum total distance perfect matching between different colors.

There is always an optimal solution without crossing matched pairs. If two matched chords cross, one can locally replace the two pairs without increasing the total length and while decreasing the number of crossings. After all crossings are removed, at least one edge of the circle is unused by all shortest paths of matched pairs, so the circle can be cut there and reduced to a line.

On a line, let $s_0 = 0$ and

$$s_i = s_{i-1} + \begin{cases} 1, & \text{the } i\text{-th ball is red,} \\ -1, & \text{otherwise.} \end{cases}$$

Then the answer is $\sum_i |s_i|$.

For the original circle, the cut position corresponds to choosing an offset x with $\min s \le x \le \max s$, so the answer is

$$\min_x \sum_i |s_i - x|.$$

The minimizing x is a median. Equivalently, the answer is

$$(\text{sum of the largest } M \text{ values among } s_i) - (\text{sum of the smallest } M \text{ values among } s_i).$$

The remaining query problem can be handled by parallel binary search with a Fenwick Tree, giving $O(Q \log^2 N)$ time. Wavelet-matrix or Mo's-algorithm variants also work.

Problem E. Circular Convolution 2

No analysis was provided in the source editorial at the time used.

Problem F. Even Circuit

Consider first whether the answer can be at most 4. This asks whether there exist four distinct indices x, y, z, w such that

$$A_x \oplus A_y \oplus A_z \oplus A_w = 0,$$

which is equivalent to

$$A_x \oplus A_y = A_z \oplus A_w.$$

Thus the case $K = 4$ can be solved by inspecting pairwise xor values and applying the pigeonhole principle.

The same idea generalizes. The condition that the answer is at most K is equivalent to the existence of two different sets of $K/2$ indices whose xor sums are equal. Since $A_i < 2^{22}$, one can show that the answer is at most 24. A bounded search over these small subset sizes, combined with the pigeonhole principle, gives about

$$O((N + \max A) \log \max A)$$

time.

Problem G. Master of Cards

Construct a graph G with vertices $1, 2, \ldots, 3N$. For every i, add three edges of color i:

- between a_i and $N + b_i$;
- between $N + b_i$ and $2N + c_i$;
- between $2N + c_i$ and a_i.

The original problem becomes the following graph problem: repeatedly select two edges that share a vertex, remove them, and output their colors. The equivalence follows from the way each original card contributes a colored triangle.

This graph operation can be greedily performed with a graph traversal such as DFS, always pairing incident edges at a vertex as they are discovered. The whole construction and traversal are linear.

The time complexity is $O(N)$.

Problem H. Shortcut on Tree

Root the tree at the given root. Color all non-root vertices from the leaves upward:

- a leaf is colored red;
- a non-leaf vertex is colored blue if it has at least one red child, and red otherwise.

Then add the following $N - 1$ directed edges:

- from every red vertex to the root;
- from the root to every blue vertex.

This construction works because from any vertex the root is reachable using at most two directed edges, and from the root any vertex is reachable using at most two directed edges. These two statements are checked directly from the red/blue definition.

Problem I. Ethanol

First solve a discretized version in which every transfer amount is exactly Δ; the original problem is obtained by taking the limit $\Delta \to 0$.

Focus on container 1. There are X/Δ operations that move mixture from container 0 into container 1, and X/Δ operations that move mixture from container 1 into container 2. The transferred concentrations may be assumed non-increasing over time. Two types of moves are never useful: adding a strictly lower concentration while container 1 still contains a higher one, and moving out before all currently higher possible inflow has been taken.

Thus the order around container 1 is forced: keep taking from container 0 while its concentration is at least the current concentration in container 1; when the inequality reverses, move as much as possible from container 1 to container 2; then alternate. The same reasoning applies successively to containers $2, 3, \ldots, N$.

After passing to the limit, concentration changes are described by differential equations. The resulting functions are linear combinations of constants and terms of the form $x^n e^{-x}$, and switching points are found by binary search.

The time complexity is $O(N^3 \cdot \text{binary search cost})$.

Problem J. International Olympiad in Fishing

Assume the first button pressed is Rank; pressing Rank again is useless, so the meaningful sequence of buttons alternates Rank, Suit, Rank, Suit, and so on. The resulting permutations of card indices become

$$p, q, r, q, r, q, r, \ldots,$$

so only the triple (p, q, r) matters. If L different triples are possible, the answer is

$$\frac{L}{N^{2N}}.$$

Indeed, if a triple has x initial configurations leading to it, its probability is x/N^{2N}, but after observing it the best success probability is $1/x$.

For fixed p, q, the permutation r is obtained by splitting p at some positions and sorting each segment by q. Every descent $p_i > p_{i+1}$ must be a split. For any other split between segments A and B, the split is canonical only if

$$\max_{a \in A} q_a^{-1} > \min_{b \in B} q_b^{-1};$$

otherwise the two segments could be merged without changing r.

Swap the order of summation: enumerate the segment partitions of the index order and of the q^{-1} order. Let S_i, T_i be the two sets associated with segment i, with equal sizes and forming partitions of $\{1, \ldots, N\}$. The condition becomes

$$\max(S_{i-1}) > \min(S_i) \quad \text{or} \quad \max(T_{i-1}) > \min(T_i).$$

Using inclusion-exclusion, define f_n for the bad contiguous blocks and g_n for valid pairs of partitions. The recurrences are

$$f_0 = g_0 = 1,$$

$$f_i = -\sum_{j=1}^{i} f_{i-j}\, j!,$$

$$g_i = -\sum_{j=1}^{i} g_{i-j} f_j \binom{i}{j}^2.$$

Then g_N is the required count L. The recurrence gives an $O(N^2)$ algorithm, and FFT plus polynomial inversion improves it to $O(N \log N)$.

Problem K. Degree Sequence 3

No detailed analysis was provided in the source editorial at the time used. The source only notes that the judges had both an $O(n \log n)$ solution and an easier $O(n \log^2 n)$ implementation.

Problem L. Bot Friends 2

No analysis was provided in the source editorial at the time used.

Problem M. Traveling in Cells 3

Assume without loss of generality that $b \le a$. The minimum cost between two points is obtained by first moving to one of the nearest points congruent to the destination coordinate modulo a, and then jumping to the destination. Thus partition the number line into blocks of length a.

By an adjustment argument, the optimal destination lies either in the block containing the median of the input points or in one of the two adjacent blocks. Enumerate these $O(1)$ candidate blocks. For a block $[L, L + a]$ and a point x_i, let $r = x_i \bmod a$. The block is split at $L + r$. On the left side, x_i only chooses between $L + r$ and $L + r - a$; on the right side, it only chooses between $L + r$ and $L + r + a$. The breakpoint between the two choices is easy to compute.

Therefore each point contributes $O(1)$ piecewise-linear functions over the destination coordinate. Sort all breakpoints and process them offline to find the minimum total cost.

The time complexity is $O(n \log n)$.